```
1. Check brackets
2. Convert and evaluate
3. Stack demo
0. Exit
Choice: 2
Expression: (12 + 3) * 2
Input: (12 + 3) * 2
Bracket status: Balanced
Postfix: 12 3 + 2 *
Result: 30
```

```
1. Check brackets
2. Convert and evaluate
3. Stack demo
0. Exit
Choice: 2
Expression: 8 + 2 * 5
Input: 8 + 2 * 5
Bracket status: Balanced
Postfix: 8 2 5 * +
Result: 18
```

```
1. Check brackets
2. Convert and evaluate
3. Stack demo
0. Exit
Choice: 2
Expression: 20 / 5 / 2
Input: 20 / 5 / 2
Bracket status: Balanced
Postfix: 20 5 / 2 /
Result: 2
```

```
1. Check brackets
2. Convert and evaluate
3. Stack demo
0. Exit
Choice: 2
Expression: 10 - 3 - 2
Input: 10 - 3 - 2
Bracket status: Balanced
Postfix: 10 3 - 2 -
Result: 5
```

```
1. Check brackets
2. Convert and evaluate
3. Stack demo
0. Exit
Choice: 2
Expression: {[2 + 3] * (4 + 1)}
Input: {[2 + 3] * (4 + 1)}
Bracket status: Balanced
Postfix: 2 3 + 4 1 + *
Result: 25
```

```
1. Check brackets
2. Convert and evaluate
3. Stack demo
0. Exit
Choice: 2
Expression: 7 / 2
Input: 7 / 2
Bracket status: Balanced
Postfix: 7 2 /
Result: 3.5
```

```
1. Check brackets
2. Convert and evaluate
3. Stack demo
0. Exit
Choice: 2
Expression: 0
Input: 0
Bracket status: Balanced
Postfix: 0
Result: 0
```

```
1. Check brackets
2. Convert and evaluate
3. Stack demo
0. Exit
Choice: 2
Expression: ([)]
Input: ([)]
Bracket status: Invalid (Mismatched brackets)
```

```
1. Check brackets
2. Convert and evaluate
3. Stack demo
0. Exit
Choice: 2
Expression: (2 + 3
Input: (2 + 3
Bracket status: Invalid (Unclosed opening bracket)
```

```
1. Check brackets
2. Convert and evaluate
3. Stack demo
0. Exit
Choice: 2
Expression: 2 + 3)
Input: 2 + 3)
Bracket status: Invalid (Unmatched closing bracket)
```

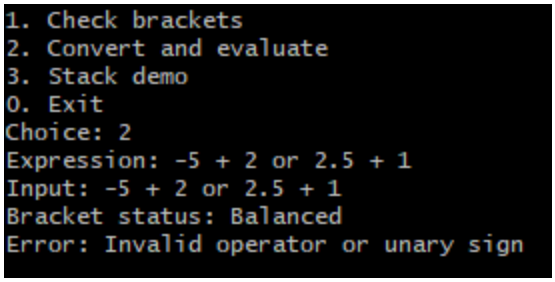
```
1. Check brackets
2. Convert and evaluate
3. Stack demo
0. Exit
Choice: 2
Expression: 5 / (3 - 3)
Input: 5 / (3 - 3)
Bracket status: Balanced
Postfix: 5 3 3 - /
Error: Division by zero
```

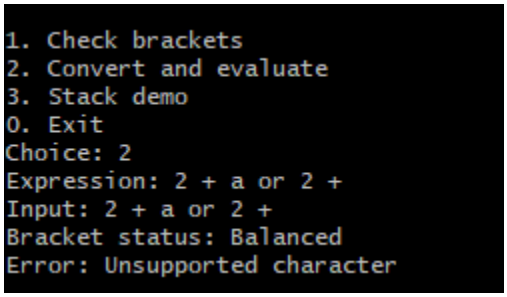
```
1. Check brackets
2. Convert and evaluate
3. Stack demo
0. Exit
Choice: 2
Expression: 2 + * 3
Input: 2 + * 3
Bracket status: Balanced
Error: Invalid operator or unary sign
```

```
1. Check brackets
2. Convert and evaluate
3. Stack demo
0. Exit
Choice: 2
Expression: 2 3 + 4
Input: 2 3 + 4
Bracket status: Balanced
Error: Missing operator
```

```
1. Check brackets
2. Convert and evaluate
3. Stack demo
0. Exit
Choice: 2
Expression: 2(3 + 4)
Input: 2(3 + 4)
Bracket status: Balanced
Error: Implicit multiplication
```

```
1. Check brackets
2. Convert and evaluate
3. Stack demo
0. Exit
Choice: 2
Expression: () or empty input
Input: () or empty input
Bracket status: Balanced
Error: Empty group or missing operand
```





19. Why does LIFO correctly match nested brackets?

In nested brackets like {[()]}, the innermost bracket opened last must be closed first before the outer brackets can close. Because a stack follows Last In, First Out (LIFO), the most recent opening bracket always sits at the top, so when a closing bracket arrives, popping the top lets us check if it matches the exact bracket that was just opened.

20. Why must evaluation pop the right operand before the left operand?

In a postfix expression like 10 2 -, the left operand (10) is pushed onto the stack first, and the right operand (2) is pushed second on top of it. Because LIFO removes the top item first, the first pop() always gets the right operand (2) and the second pop() gets the left operand (10), ensuring non-commutative operations like 10 - 2 = 8 and 20 / 5 = 4 compute in the right order.

21. What is the difference between balanced brackets and a valid arithmetic expression?

Balanced brackets only check that every opening bracket has a matching closing bracket in the proper nested order, ignoring numbers and operators. A valid arithmetic expression must also follow math syntax rules, such as having operands in the right places, no consecutive operators (2 + * 3), no empty groups (()), and no division by zero.

22. Compare your stack ADT with the function call stack. In a nested function call, which call returns first?

Both stacks follow LIFO, but our Stack<T> ADT is manually managed by our code using new and delete to store data like characters and numbers, whereas the function call stack is automatically

managed by the computer's runtime to track active functions and return addresses. In a nested call like main() -> solve() -> helper(), the innermost call (helper()) finishes and returns first.

23. What memory is allocated and released by your implementation? Explain one resizing or node-deletion step.

Our linked-list stack allocates a Node on the heap using new Node(...) during every push(), and releases each node using delete during pop() and in the destructor ~Stack() so no memory leaks occur. During a node-deletion step in pop(), we save the current head pointer in a temporary pointer tmp, copy its stored value, move head to head->next, and call delete tmp to free that node's memory.